UNIT 5 · TOPIC 5.4

Asymmetric Cryptography

Assignment, Answer Key

Teacher answer key, 35 points total.

Practice Questions

5.4.a · Public/private key pairs

1. Alice wants to send Bob a message that only Bob can read. Which key does Alice use to encrypt the message? (1 pt)

- ● **A. Bob's public key**
- ○ B. Alice's public key
- ○ C. Alice's private key
- ○ D. Bob's private key

Why: Encrypt with the recipient's public key; only the matching private key (held by Bob) can decrypt it.

2. Bob wants to prove a document came from him. Which key does he use to sign it, and which key does a verifier use to check the signature? (1 pt)

- ○ A. Sign with his public key; verify with his private key
- ● **B. Sign with his private key; verify with his public key**
- ○ C. Sign and verify both use his private key
- ○ D. Sign and verify both use his public key

Why: Signing uses the private key (only Bob holds it); verification uses the corresponding public key, which anyone can have.

3. Given primes p = 3 and q = 11, select ALL statements that are true about the resulting RSA parameters. (2 pts)

- ■ **A. n = p * q = 33**
- ■ **B. phi(n) = (p-1)(q-1) = 20**
- □ C. n = 20
- □ D. phi(n) = 33

Why: n = 3*11 = 33, and phi(n) = (3-1)*(11-1) = 2*10 = 20.

4. In one sentence, explain why an attacker cannot derive the private key just by knowing the public key (e, n). (1 pt)

Acceptable answers: factor; factoring; hard to factor; cannot factor n; computationally infeasible to factor

Why: Recovering d requires knowing phi(n), which requires factoring n back into p and q, computationally infeasible for large n.

5.4.b · RSA algorithm

5. Which formula is the RSA encryption operation? (1 pt)

- ○ A. c = m * e mod n
- ○ B. c = m^d mod n
- ● **C. c = m^e mod n**
- ○ D. c = e^m mod n

Why: Encryption raises the message to the power e (the public exponent) and reduces mod n: c = m^e mod n.

6. Given p = 3, q = 11, e = 7, what is the private exponent d? (1 pt)

○ A. 7

● B. 3

○ C. 13

○ D. 20

Why: phi(n) = (3-1)(11-1) = 20. d is the modular inverse of 7 mod 20: (7*3) mod 20 = 21 mod 20 = 1, so d = 3.

7. Using n = 33, e = 7, d = 3 (from p=3, q=11), select ALL true statements. (2 pts)

■ A. Encrypting m = 5 gives c = 14

■ B. Decrypting c = 14 with d = 3 gives back m = 5

■ C. The message m must satisfy 0 <= m < 33

□ D. d can be any integer as long as it differs from e

Why: 5^7 mod 33 = 14, and 14^3 mod 33 = 5, confirming the round trip. m must be < n. d is not arbitrary, it must satisfy e*d ≡ 1 (mod phi(n)).

8. In one sentence, why is e = 65537 the near-universal choice for the public exponent in real RSA implementations? (1 pt)

Acceptable answers: prime; fast; efficient; low-exponent attack; 1 bit; few bits set; small attack

Why: It is prime (so it is coprime with phi(n) for essentially any p, q), its binary form makes modular exponentiation fast via repeated squaring, and it is large enough to resist low-exponent attacks that break very small e values.

5.4.c · Digital signatures

9. Bob wants to digitally sign a document so others can verify it came from him. Which key does he sign with, and which key do verifiers use? (1 pt)

○ A. Sign with his public key; verify with his private key

○ B. Sign and verify both use a shared secret key

○ C. Sign and verify both use his private key

● D. Sign with his private key; verify with his public key

Why: Only Bob holds his private key, so signing with it is what lets anyone with his public key confirm the signature came from him.

10. Why do real signature schemes hash the message first instead of running RSA directly on the full message? (1 pt)

○ A. Hashing makes the message secret so only the signer can read it

● B. It produces a fixed-size input, is much faster than signing large data directly, and avoids structural attacks possible when signing raw messages

○ C. RSA cannot process any input longer than 256 bits under any circumstance

○ D. Hashing is required so the signature can later be decrypted back into the original message

Why: Hash-then-sign keeps the expensive RSA operation to a single fixed-size digest, which is both faster and avoids algebraic forgery attacks possible when signing arbitrary raw messages directly.

11. A message was signed, and verification against the signer's correct public key returns True. Select ALL guarantees this gives the verifier. (2 pts)

■ A. Authenticity, only the private-key holder could have produced this signature

■ B. Integrity, the message was not altered after signing

□ C. Confidentiality, no one else could have read the message in transit

■ **D. Non-repudiation, the signer cannot credibly deny having signed it**

Why: A valid signature proves authenticity, integrity, and non-repudiation, but not confidentiality, the message itself is not encrypted by the signing process.

12. In one sentence, explain why HMAC (5.3.c) cannot provide non-repudiation the way a digital signature can. (1 pt)

Acceptable answers: shared secret; shared key; both parties know the key; either party could have produced it; symmetric key

Why: HMAC uses one shared secret key that both parties hold, so a valid tag proves only that 'someone with the key' produced it, either party could have, so neither can be singled out as the signer. A digital signature uses a private key only the signer holds, which removes that ambiguity.

13. Alice receives a message and a claimed signature. She verifies using Mallory's public key by mistake, and verification succeeds. What does this tell Alice? (1 pt)

● **A. Nothing trustworthy about the intended sender, she has only confirmed Mallory's private key produced this signature, not the sender she meant to check**

○ B. The message definitely came from the person she intended and was not tampered with

○ C. The message was encrypted and only she can read it

○ D. The signature is invalid and she should reject the message

Why: Verification only proves a signature matches whichever public key you check it against. Using the wrong public key means a successful verification tells you nothing about the sender you actually intended to authenticate.

5.4.d · PKI and digital certificates

14. What does a Certificate Authority (CA) actually vouch for when it signs a certificate? (1 pt)

● **A. That the subject's public key really belongs to the identity named in the certificate**

○ B. That the website's content is safe and free of malware

○ C. That the subject's private key is strong enough to resist brute-force attacks

○ D. That the website will remain online for the full validity period

Why: A certificate binds a public key to a claimed identity, verified and signed by the CA. It says nothing about the site's content or safety.

15. In a certificate chain leaf -> intermediate -> root, how is the leaf certificate's signature verified? (1 pt)

● **A. Using the intermediate CA's public key**

○ B. Using the leaf's own public key

○ C. Using the root CA's public key directly, skipping the intermediate

○ D. Using a shared secret between the leaf and the browser

Why: Each certificate's signature is verified with the public key of whoever issued (signed) it -- for the leaf, that's the intermediate CA.

16. Select ALL statements that are true about root CAs and trust stores. (2 pts)

■ **A. A root CA's certificate is self-signed (issuer equals subject).**

□ B. A browser trusts a root CA because the root's certificate says so.

■ **C. The set of trusted root public keys is curated by the OS/browser vendor, not derived from the chain itself.**

□ D. Root CAs typically sign end-entity (leaf) certificates directly rather than delegating to intermediates.

Why: Roots are self-signed, and trust in them comes from being pre-installed in the OS/browser trust store -- never from anything inside the certificate chain itself. Roots typically delegate signing to intermediate CAs rather than signing leaves directly.

17. In one or two sentences, explain the difference between certificate expiration and certificate revocation. (1 pt)

Acceptable answers: scheduled; before the expiration date; compromise; private key compromise; unscheduled; crl; ocsp

Why: Expiration is a scheduled, built-in end to a certificate's validity window (not_after). Revocation is an unscheduled, active withdrawal of trust before that date -- typically because the private key was compromised -- checked via a CRL or OCSP.

18. A server presents a self-signed certificate that is not present in your browser's trust store. The certificate's own signature verifies correctly. What should happen? (1 pt)

○ A. Trust it -- a verifying signature is sufficient

○ B. Trust it, but only for encrypted traffic, not for identity

○ C. Trust it automatically since it is the root of its own chain

● D. Reject it -- no independent third party vouches for the binding, regardless of whether the self-signature verifies

Why: A self-signed certificate can have a perfectly valid signature (it signed itself) and still be untrustworthy -- trust requires being anchored in a trust store the client already relies on, not just a passing signature check.

Coding Assignments (Python 3)

5.4.a · Public/private key pairs

3 pts · auto-graded on the site

Coding problem, 5.4.a: RSA key-pair parameters

Skill: Mitigate Risk · **Maps to:** 5.4.A · **Language:** Python 3 (pytest)

Problem

Before RSA can encrypt or sign anything, it needs a key pair. Given two prime numbers p and q, implement the first steps of RSA key generation described in `atom.md`: compute the modulus n, compute phi(n), and validate whether a proposed public/private exponent pair (e, d) actually satisfies the required mathematical relationship.

Function contract

Implement, in `starter.py`:

- `rsa_params(p: int, q: int) -> tuple`, given two (small, distinct) primes, return `(n, phi_n)` where `n = p * q` and `phi_n = (p - 1) * (q - 1)`.
- `is_valid_keypair(e: int, d: int, phi_n: int) -> bool`, return `True` iff `(e * d) % phi_n == 1`, i.e. d is the modular inverse of e mod phi_n.
- `solve(data: dict) -> dict`, the blackbox entry point that dispatches to the two functions above (see `interface.md` / `blackbox.yaml`).

The math (see atom.md)

```
n       = p * q
phi(n)  = (p - 1) * (q - 1)
valid keypair (e, d)  <=>  (e * d) mod phi(n) == 1
```

The public key is `(e, n)`; the private key is `(d, n)`. Both share n, but d cannot be recovered from e and n alone without knowing `phi(n)` (which requires factoring n back into p and q).

Input / Output

- `rsa_params(p, q)`: `p, q: int` (primes) -> `(n, phi_n)` tuple of `int`.
- `is_valid_keypair(e, d, phi_n)`: all `int` -> `bool`.
- `solve(data)`: `data = {"op": "params", "p": <int>, "q": <int>}` -> `{"n": <int>, "phi_n": <int>}`, or `data = {"op": "validate", "e": <int>, "d": <int>, "phi_n": <int>}` -> `{"valid": <bool>}`.

Examples

```
rsa_params(3, 11)  -> (33, 20)     # n = 3*11 = 33, phi(n) = 2*10 = 20
rsa_params(5, 11)  -> (55, 40)     # n = 5*11 = 55, phi(n) = 4*10 = 40

is_valid_keypair(e=3, d=7, phi_n=20)  -> True   # 3*7 = 21, 21 % 20 == 1
is_valid_keypair(e=3, d=6, phi_n=20)  -> False  # 3*6 = 18, 18 % 20 != 1
```

Constraints

- Deterministic, offline, standard library only. No external crypto/bignum libraries, this problem implements the underlying arithmetic directly.
- Inputs are small, textbook-scale integers as in the atom's worked examples; no primality checking is required.
- Must pass all cases in `tests.py` (visible + hidden), including the p=3,q=11 and p=5,q=11 examples from `atom.md`, and both valid and invalid keypair checks.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def rsa_params(p: int, q: int) -> tuple:
    n = p * q
    phi_n = (p - 1) * (q - 1)
    return (n, phi_n)


def is_valid_keypair(e: int, d: int, phi_n: int) -> bool:
    return (e * d) % phi_n == 1


def solve(data: dict) -> dict:
    op = data["op"]
    if op == "params":
        n, phi_n = rsa_params(data["p"], data["q"])
        return {"n": n, "phi_n": phi_n}
    elif op == "validate":
        return {"valid": is_valid_keypair(data["e"], data["d"], data["phi_n"])}
    else:
        raise ValueError(f"unknown op: {op!r}")
```

5.4.b · RSA algorithm

3 pts · auto-graded on the site

Coding problem, 5.4.b: RSA algorithm

Skill: Mitigate Risk · **Maps to:** 5.4.A, 5.4.B · **Language:** Python 3 (pytest)

Problem

A small internal tool needs to demonstrate RSA end to end on textbook-scale numbers: derive a key pair from two primes and a chosen public exponent, encrypt a small integer message with the public key, and decrypt it back with the private key. Implement the key derivation and the modular-exponentiation encrypt/decrypt operations directly, you may use Python's built-in `pow(base, exp, mod)` for the modular exponentiation itself, but the extended Euclidean algorithm used to derive the private exponent d must be implemented by hand.

Function contract

Implement, in `starter.py`:

- `egcd(a: int, b: int) -> tuple[int, int, int]`, extended Euclidean algorithm; returns `(g, x, y)` such that `a*x + b*y == g == gcd(a, b)`.
- `mod_inverse(e: int, phi_n: int) -> int`, the modular inverse d of e mod `phi_n`, i.e. `(e * d) % phi_n == 1`. Raises `ValueError` if no inverse exists (`gcd(e, phi_n) != 1`).
- `rsa_keygen(p: int, q: int, e: int) -> dict`, derives n, `phi_n`, and d from primes p, q and public exponent e. Returns `{"n": n, "phi_n": phi_n, "e": e, "d": d}`. Raises `ValueError` if e does not satisfy `1 < e < phi_n`.
- `encrypt(m: int, e: int, n: int) -> int`, RSA encrypt: `c = m^e mod n`. Raises `ValueError` if `m` is not in `0 <= m < n`.
- `decrypt(c: int, d: int, n: int) -> int`, RSA decrypt: `m = c^d mod n`.
- `solve(data: dict) -> dict`, the blackbox entry point that dispatches to the functions above (see `interface.md` / `blackbox.yaml`).

Input / Output

- `rsa_keygen(p, q, e)`: p, q are (small, in this atom's tests) prime integers; e is the chosen public exponent -> `{"n": int, "phi_n": int, "e": int, "d": int}`.
- `encrypt(m, e, n)`: `m: int`, `e: int`, `n: int` -> ciphertext `int`.
- `decrypt(c, d, n)`: `c: int`, `d: int`, `n: int` -> plaintext `int`.
- `solve(data)`: - `{"op": "keygen", "p": int, "q": int, "e": int}` -> `{"n": int, "phi_n": int, "e": int, "d": int}` - `{"op": "encrypt", "m": int, "e": int, "n": int}` -> `{"c": int}` - `{"op": "decrypt", "c": int, "d": int, "n": int}` -> `{"m": int}` - `{"op": "roundtrip", "p": int, "q": int, "e": int, "m": int}` -> `{"n": int, "phi_n": int, "d": int, "c": int, "m_decrypted": int}`

Examples

```
rsa_keygen(3, 11, 7)          -> {"n": 33, "phi_n": 20, "e": 7, "d": 3}
encrypt(5, 7, 33)             -> 14
decrypt(14, 3, 33)            -> 5

solve({"op": "roundtrip", "p": 3, "q": 11, "e": 7, "m": 5})
  -> {"n": 33, "phi_n": 20, "d": 3, "c": 14, "m_decrypted": 5}
```

Constraints

- Deterministic, offline, standard library only (`pow()` for modular exponentiation is allowed and expected; do not import any external crypto library, the extended Euclidean algorithm and key derivation must be your own code).

- Must pass all cases in `tests.py` (visible + hidden), including edge cases such as `m = 0`, `m` at or beyond `n` (rejected), and `e` values that are not coprime with `phi_n` (rejected).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def egcd(a: int, b: int):
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)


def mod_inverse(e: int, phi_n: int) -> int:
    g, x, _ = egcd(e, phi_n)
    if g != 1:
        raise ValueError(f"e={e} has no modular inverse mod phi_n={phi_n} (gcd={g})")
    return x % phi_n


def rsa_keygen(p: int, q: int, e: int) -> dict:
    n = p * q
    phi_n = (p - 1) * (q - 1)
    if not (1 < e < phi_n):
        raise ValueError(f"e={e} must satisfy 1 < e < phi_n={phi_n}")
    d = mod_inverse(e, phi_n)
    return {"n": n, "phi_n": phi_n, "e": e, "d": d}


def encrypt(m: int, e: int, n: int) -> int:
    if not (0 <= m < n):
        raise ValueError(f"message m={m} must satisfy 0 <= m < n={n}")
    return pow(m, e, n)


def decrypt(c: int, d: int, n: int) -> int:
    return pow(c, d, n)


def solve(data):
    op = data["op"]
    if op == "keygen":
        return rsa_keygen(data["p"], data["q"], data["e"])
    elif op == "encrypt":
        return {"c": encrypt(data["m"], data["e"], data["n"])}
    elif op == "decrypt":
        return {"m": decrypt(data["c"], data["d"], data["n"])}
    elif op == "roundtrip":
        keys = rsa_keygen(data["p"], data["q"], data["e"])
        c = encrypt(data["m"], keys["e"], keys["n"])
        m_decrypted = decrypt(c, keys["d"], keys["n"])
        return {
            "n": keys["n"],
            "phi_n": keys["phi_n"],
            "d": keys["d"],
```

```
            "c": c,
            "m_decrypted": m_decrypted,
        }
    else:
        raise ValueError(f"unknown op: {op!r}")
```

5.4.c · Digital signatures

3 pts · auto-graded on the site

Coding problem, 5.4.c: Implement RSA digital signatures (hash-then-sign)

Skill: Mitigate Risk · **Maps to:** 5.4.B · **Language:** Python 3 (pytest)

Problem

A software vendor wants to let users verify that a downloaded update really came from them and was not modified in transit. Rather than signing the whole update file directly with RSA (slow, and structurally risky per atom.md), implement the standard "hash, then sign" digital signature scheme: hash the message with SHA-256, reduce the digest mod n, and apply the RSA private-key operation to that digest, not the raw message.

Function contract

Implement, in `starter.py`:

- `sign(message: bytes, d: int, n: int) -> int`, return the signature `(SHA-256(message) mod n)^d mod n`, using the signer's private exponent d and modulus n.
- `verify(message: bytes, signature: int, e: int, n: int) -> bool`, recompute `SHA-256(message) mod n` and compare it to `signature^e mod n`, using the signer's public exponent e and modulus n. Return `True` iff they match.
- `solve(data: dict) -> dict`, the blackbox entry point that dispatches to the two functions above (see `interface.md` / `blackbox.yaml`).

You may use `hashlib.sha256` for the hash step only. Implement the digest-reduction and the modular exponentiation (`pow(base, exp, mod)` is fine, it's Python's built-in, not an external crypto library) yourself, per the construction below.

The construction

```
digest        = int.from_bytes(SHA256(message).digest(), "big") % n
sign:         signature = digest^d mod n
verify:       recovered = signature^e mod n     ; valid iff recovered == digest (recomputed)
```

- Hashing first keeps the signature a fixed size regardless of message length, and means a single changed byte in `message` produces a completely different digest (avalanche effect, 5.3.b), which is what makes tampering detectable.
- d and n form the private key; e and n form the matching public key (5.4.b).

Input / Output

- `sign(message, d, n)`: `message: bytes`, `d: int`, `n: int` -> `int` signature.
- `verify(message, signature, e, n)`: `signature: int`, `e: int`, `n: int` -> `bool`.
- `solve(data)`: `data = {"op": "sign", "message": "<str>", "d": <int>, "n": <int>}` -> `{"signature": <int>}`, or `data = {"op": "verify", "message": "<str>", "signature": <int>, "e": <int>, "n": <int>}` -> `{"valid": <bool>}`.

Examples

Using the textbook-scale key pair p=61, q=53 -> n=3233, e=17, d=2753:

```
sign(b"pay Bob $50", 2753, 3233)        -> some int in [0, 3233)
verify(b"pay Bob $50", <that signature>, 17, 3233)   -> True
verify(b"pay Bob $950", <that signature>, 17, 3233)  -> False   # message tampered after signing
```

Constraints

- Deterministic, offline, standard library only (`hashlib` only, no external crypto packages).
- Correctness is checked in `tests.py` against an independent oracle that recomputes the same hash-then-sign math directly (not by importing your functions).
- Must pass all cases in `tests.py` (visible + hidden), including: a tampered message failing verification, a wrong public key failing verification, an empty message, and determinism (signing the same message twice yields the same signature).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import hashlib


def _digest_mod_n(message: bytes, n: int) -> int:
    digest_bytes = hashlib.sha256(message).digest()
    digest_int = int.from_bytes(digest_bytes, "big")
    return digest_int % n


def sign(message: bytes, d: int, n: int) -> int:
    digest = _digest_mod_n(message, n)
    return pow(digest, d, n)


def verify(message: bytes, signature: int, e: int, n: int) -> bool:
    expected_digest = _digest_mod_n(message, n)
    recovered_digest = pow(signature, e, n)
    return expected_digest == recovered_digest


def solve(data: dict) -> dict:
    op = data["op"]
    message = data["message"].encode("utf-8")
    if op == "sign":
        signature = sign(message, data["d"], data["n"])
        return {"signature": signature}
    elif op == "verify":
        valid = verify(message, data["signature"], data["e"], data["n"])
        return {"valid": valid}
    else:
        raise ValueError(f"unknown op: {op!r}")
```

5.4.d · PKI and digital certificates

4 pts · auto-graded on the site

Coding problem, 5.4.d: Validate a certificate chain

Skill: Mitigate Risk · **Maps to:** 5.4.B, 5.4.C · **Language:** Python 3 (pytest)

Problem

A browser has just received a TLS certificate chain from a server: a leaf (end-entity) certificate and one or more issuing CA certificates, and it needs to decide whether to trust it. Real browsers call into a full X.509 library to do this; you are going to implement the *validation logic itself*, the same logic that library runs under the hood, using the small-number RSA sign/verify math from 5.4.b/5.4.c.

Each certificate is a plain `dict`:

```
{
  "subject": "www.meridianbank.example",     # who this certificate identifies
  "issuer": "Meridian Intermediate CA",       # who issued (signed) it
  "public_key": [17, 2501],                   # [e, n] -- this cert's own public key
  "signature": 1271,                          # issuer's signature over this cert's fields
  "not_before": 1000,                         # validity window, integer "time units"
  "not_after": 9000,
  "revoked": False,
}
```

A **chain** is a list of these dicts ordered leaf-first: `[leaf, intermediate..., root]`. The last entry is claimed to be a root CA certificate (self-signed: its own `issuer` equals its own `subject`).

What you implement (in `starter.py`)

- `compute_digest(cert: dict) -> int`, a deterministic, keyless digest of a certificate's identity fields (everything except `signature` and `revoked`), used as the thing that actually gets signed. Canonicalize as the string `f"{subject}|{issuer}|{public_key[0]}:{public_key[1]}|{not_before}|{not_after}"` and reduce it with the rolling hash: ``python h = 0 for ch in s: h = (h * 131 + ord(ch)) % (2**61 - 1)`

- `rsa_sign(digest: int, d: int, n: int) -> int`, `signature = (digest mod n) ** d mod n`.

- `rsa_verify(digest: int, signature: int, e: int, n: int) -> bool`, return whether `signature ** e mod n == digest mod n`.

- `validate_chain(chain: list[dict], trusted_roots: dict[str, list[int]], current_time: int) -> dict`, validate the whole chain (see below). `trusted_roots` maps a trusted root CA's `subject` name to its known-good `[e, n]` public key (this is the browser's local trust store, never taken from the chain itself).

- `solve(data: dict) -> dict`, blackbox dispatcher (see `interface.md`).

Validation algorithm

Walk the chain from the leaf (index 0) upward. For each certificate `cert` at index `i`:

1. **Validity window**, if `current_time < cert["not_before"]` or `current_time > cert["not_after"]` -> reason `"expired"`. 2. **Revocation**, if `cert["revoked"]` is `True` -> reason `"revoked"`. 3. **Chain linkage / signer's public key:** - If this is *not* the last certificate in the chain, the next certificate up (`chain[i+1]`) must be its issuer: `cert["issuer"] == chain[i+1]["subject"]`. If not -> reason `"broken_chain"`. The public key used to check this cert's signature is `chain[i+1]["public_key"]`. - If this *is* the last certificate in the chain, it must be a certificate for a root in `trusted_roots`: `cert["subject"]` must be a key in `trusted_roots`, and `trusted_roots[cert["subject"]]` must equal `cert["public_key"]` exactly. If either check fails -> reason `"untrusted_root"`. The public key used to check this cert's own (self-)signature is `trusted_roots[cert["subject"]]`. 4. **Signature check**, recompute `compute_digest(cert)` and check `rsa_verify(digest, cert["signature"], e, n)` using the public key established in step 3. If it fails -> reason `"bad_signature"`.

If every certificate in the chain passes all four checks, the chain is valid -> reason `"valid"`. Stop at the *first* certificate (starting from the leaf) that fails any check and return that specific reason -- do not keep scanning past a failure.

Input / Output

- `validate_chain(chain, trusted_roots, current_time)` -> `{"valid": bool, "reason": str}` where `reason` is one of: `"valid"`, `"expired"`, `"revoked"`, `"broken_chain"`, `"untrusted_root"`, `"bad_signature"`.

- `solve(data)`: `data = {"op": "validate", "chain": [...], "trusted_roots": {...}, "current_time": int}` -> `validate_chain(...)`'s return dict directly. `data = {"op": "digest", "cert": {...}}` -> `{"digest": int}`.

Examples

```
validate_chain([leaf, intermediate, root], {"Meridian Root CA": [17, 2773]}, 5000)
  -> {"valid": True, "reason": "valid"}

validate_chain([expired_leaf, intermediate, root], {"Meridian Root CA": [17, 2773]}, 9500)
  -> {"valid": False, "reason": "expired"}
```

Constraints

- Deterministic, offline, standard library only. No `cryptography`, no `ssl`, no `hashlib` -- this is a from-scratch textbook-RSA exercise (same style as 5.4.b/5.4.c), not a real X.509 parser.

- `trusted_roots` is the only source of truth for which root keys are trusted -- a self-signed certificate whose subject is not in `trusted_roots` must be rejected as `"untrusted_root"`, even if its self-signature verifies correctly.

- Must pass all cases in `tests.py` (visible + hidden): a fully valid chain, an expired leaf, a revoked intermediate, a broken issuer/subject link, an untrusted (self-signed but unrecognized) root, and a tampered signature.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import Any

_MODULUS = (2 ** 61) - 1


def compute_digest(cert: dict) -> int:
    e, n = cert["public_key"]
    canonical = f'{cert["subject"]}|{cert["issuer"]}|{e}:{n}|{cert["not_before"]}|{cert["not_after"]}'
    h = 0
    for ch in canonical:
        h = (h * 131 + ord(ch)) % _MODULUS
    return h


def rsa_sign(digest: int, d: int, n: int) -> int:
    return pow(digest % n, d, n)


def rsa_verify(digest: int, signature: int, e: int, n: int) -> bool:
    return pow(signature, e, n) == digest % n


def validate_chain(chain: list, trusted_roots: dict, current_time: int) -> dict:
    for i, cert in enumerate(chain):
        if current_time < cert["not_before"] or current_time > cert["not_after"]:
            return {"valid": False, "reason": "expired"}
```

```
        if cert.get("revoked", False):
            return {"valid": False, "reason": "revoked"}

        if i + 1 < len(chain):
            issuer_cert = chain[i + 1]
            if cert["issuer"] != issuer_cert["subject"]:
                return {"valid": False, "reason": "broken_chain"}
            issuer_pubkey = issuer_cert["public_key"]
        else:
            root_key = trusted_roots.get(cert["subject"])
            if root_key is None or list(root_key) != list(cert["public_key"]):
                return {"valid": False, "reason": "untrusted_root"}
            issuer_pubkey = root_key

        digest = compute_digest(cert)
        e, n = issuer_pubkey
        if not rsa_verify(digest, cert["signature"], e, n):
            return {"valid": False, "reason": "bad_signature"}

    return {"valid": True, "reason": "valid"}


def solve(data: Any) -> Any:
    op = data["op"]
    if op == "validate":
        return validate_chain(data["chain"], data["trusted_roots"], data["current_time"])
    elif op == "digest":
        return {"digest": compute_digest(data["cert"])}
    else:
        raise ValueError(f"unknown op: {op!r}")
```